

NumPy for Data Science

Day 1 — Lecture Notes & Practical Tasks

Beginner to Intermediate · Hands-On Exercises Included

1. Introduction to NumPy

NumPy (Numerical Python) is the foundational library for scientific computing in Python. It provides a high-performance multidimensional array object called ndarray and a large collection of mathematical functions to operate on these arrays efficiently.

1.1 Why NumPy?

- Python lists are slow for numerical work — they store pointers, not raw values
- NumPy arrays store homogeneous, typed data in contiguous memory → much faster
- Vectorised operations eliminate explicit Python loops
- Forms the backbone of Pandas, Matplotlib, Scikit-learn, TensorFlow, SciPy
- Native support for linear algebra, Fourier transforms, and random sampling

1.2 Python List vs NumPy Array

```
import numpy as np
import time

# Python list multiplication
py_list = [1, 2, 3, 4, 5]
print(py_list * 2)          # [1, 2, 3, 4, 5, 1, 2, 3, 4, 5]  <- REPEATS

# NumPy array multiplication
np_arr = np.array([1, 2, 3, 4, 5])
print(np_arr * 2)          # [ 2  4  6  8 10]  <- ELEMENT-WISE

# Speed benchmark
n = 1_000_000
big_list = list(range(n))
```

```
t = time.time()
squared_list = [x**2 for x in big_list]
print(f"List: {time.time()-t:.4f}s")

big_arr = np.arange(n)
t = time.time()
squared_arr = big_arr**2
print(f"NumPy: {time.time()-t:.4f}s")    # ~100x faster
```

1.3 Installing & Importing

```
# In your terminal / command prompt:
pip install numpy

# In your Python script / Jupyter notebook:
import numpy as np          # 'np' is the universal alias

# Verify
print(np.__version__)      # e.g., 1.26.4

# Useful constants
print(np.pi)              # 3.14159265...
print(np.e)               # 2.71828182...
print(np.inf)             # infinity
print(np.nan)             # Not a Number
```

2. Creating NumPy Arrays

The core object is the ndarray. You can create arrays from existing Python data or using NumPy's built-in factory functions.

2.1 From Python Lists / Tuples

```
import numpy as np

# 1D array
a = np.array([10, 20, 30, 40])
print(a)          # [10 20 30 40]
print(type(a))    # <class 'numpy.ndarray'>

# 2D array (list of lists)
m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m)

# Specify data type
b = np.array([1, 2, 3], dtype=float)
```

```
c = np.array([1.5, 2.7], dtype=int)    # truncates
print(b.dtype, c.dtype)              # float64, int64
```

2.2 Factory Functions

```
import numpy as np

np.zeros((3, 4))          # 3x4 array of 0.0
np.ones((2, 3))           # 2x3 array of 1.0
np.full((2, 2), 7)        # 2x2 array of 7
np.eye(3)                 # 3x3 identity matrix
np.empty((2, 2))          # uninitialized (random garbage)

# Range-based
np.arange(10)              # [0 1 2 ... 9]
np.arange(0, 20, 3)        # [0 3 6 9 12 15 18]
np.linspace(0, 1, 5)       # [0. 0.25 0.5 0.75 1.]
np.logspace(0, 2, 3)       # [1. 10. 100.]

# Random arrays
np.random.rand(3, 3)       # uniform [0,1)
np.random.randn(3, 3)      # standard normal
np.random.randint(1, 10, size=(3, 3))
```

💡 *Note: np.arange uses a step value; np.linspace uses the number of points. Use linspace when exact endpoint inclusion matters.*

2.3 Tasks — Creating Arrays

Task 2.3.1: Create a 1D array of integers from 5 to 50 in steps of 5. Print its shape and dtype.

Task 2.3.2: Create a 4x4 matrix of zeros and then set the diagonal elements to 1 (without using np.eye). Hint: use indexing with range().

Task 2.3.3: Generate an array of 20 evenly-spaced values between 0 and 2π (np.pi). Compute sin of each value.

Task 2.3.4: Create a 3x3 random integer array with values between 10 and 99. Find its minimum and maximum values.

Task 2.3.5: Build a 5x5 checkerboard pattern (alternating 0s and 1s) using slicing and zeros.

3. Array Attributes

```
import numpy as np

a = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 10, 11, 12]])
```

```
print(a.shape)      # (3, 4)      - (rows, cols)
print(a.ndim)       # 2          - number of axes
print(a.size)       # 12         - total elements
print(a.dtype)      # int64
print(a.itemsize)   # 8          - bytes per element
print(a.nbytes)     # 96         - total bytes

# Change dtype
b = a.astype(np.float32)
print(b.dtype)      # float32 (uses 4 bytes per element)
print(b.nbytes)     # 48

# Transpose
print(a.T.shape)    # (4, 3)
```

3.1 Common dtypes

- int8 / int16 / int32 / int64 → signed integers (8–64 bit)
- uint8 / uint16 ... → unsigned integers (no negatives)
- float32 / float64 → floating-point (single/double precision)
- complex64 / complex128 → complex numbers
- bool → True/False stored as single bit
- U20 / S20 → Unicode / byte string of length 20

3.2 Tasks — Attributes

Task 3.2.1: Create a (4, 3, 2) array of ones. Print its ndim, shape, size, and total memory in bytes.

Task 3.2.2: Create an int64 array and convert it to float32. Compare nbytes before and after.

Task 3.2.3: Write a function describe(arr) that prints all six attributes of any given array.

4. Indexing & Slicing

NumPy offers several indexing strategies. Understanding views vs. copies is critical to avoid subtle bugs.

4.1 1D Indexing

```
import numpy as np

a = np.array([10, 20, 30, 40, 50, 60])

print(a[0])        # 10
print(a[-1])       # 60
print(a[1:4])      # [20 30 40]
print(a[::2])      # [10 30 50] every other
print(a[::-1])     # [60 50 40 30 20 10] reversed
```

```
# VIEWS – modifying slice modifies original
s = a[2:5]
s[0] = 99
print(a)          # [10 20 99 40 50 60]

# COPY – safe modification
c = a[2:5].copy()
c[0] = 0
print(a)          # [10 20 99 40 50 60]  unchanged
```

4.2 2D Indexing

```
import numpy as np

m = np.arange(1, 13).reshape(3, 4)
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]

print(m[0, 2])      # 3      (row 0, col 2)
print(m[1, :])      # [5 6 7 8]   row 1
print(m[:, 2])      # [3 7 11]   col 2
print(m[0:2, 1:3])  # [[2 3],[6 7]] sub-matrix
print(m[-1, -1])    # 12   last element
```

4.3 Boolean Indexing

```
import numpy as np

a = np.array([15, 42, -3, 87, -12, 55])

mask = a > 0
print(mask)          # [True True False True False True]
print(a[mask])       # [15 42 87 55]

# Inline
print(a[a > 40])      # [42 87 55]

# np.where
print(np.where(a > 0, a, 0))    # clip negatives → 0
print(np.where(a > 50, 'High', 'Low'))
```

4.4 Tasks — Indexing & Slicing

Task 4.4.1: Given `arr = np.arange(1, 26).reshape(5, 5)`, extract the 3×3 sub-matrix from the centre.

Task 4.4.2: Reverse every row of a 4×4 random integer matrix using slicing.

Task 4.4.3: From the array `[34, -5, 12, 0, -8, 99, 47, -1]`, extract all negative values and replace them with their absolute values in the original array.

Task 4.4.4: Create a 6×6 identity-like matrix using `np.eye`, then set all values in rows 2–4 and columns 2–4 to 5.

Task 4.4.5: Use `np.where` to classify an array of 10 random scores (0–100): 'A' if ≥ 80 , 'B' if ≥ 60 , else 'C'.

5. Array Operations & Universal Functions

5.1 Element-wise Arithmetic

```
import numpy as np

a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

print(a + b)      # [11 22 33 44]
print(b - a)      # [ 9 18 27 36]
print(a * b)      # [10 40 90 160]
print(b / a)      # [10. 10. 10. 10.]
print(b // a)     # [10 10 10 10] floor div
print(b % 3)      # [1 2 0 1]
print(a ** 2)     # [1 4 9 16]
```

5.2 Universal Functions (ufuncs)

```
import numpy as np

a = np.array([1, 4, 9, 16])

np.sqrt(a)        # [1.  2.  3.  4.]
np.square(a)      # [1 16 81 256]
np.abs(np.array([-1, -2, 3])) # [1 2 3]
np.exp(np.array([0, 1, 2]))  # [1. e e^2]
np.log(a)         # natural log
np.log2(a)        # base-2 log
np.log10(a)       # base-10 log
np.sin(np.linspace(0, np.pi, 5))
np.ceil(np.array([1.2, 3.7])) # [2. 4.]
np.floor(np.array([1.9, 3.1])) # [1. 3.]
np.round(np.array([1.56, 3.44]), 1) # [1.6 3.4]
```

5.3 Aggregate Functions

```
import numpy as np

m = np.array([[1, 2, 3],
              [4, 5, 6]])
```

```
m.sum()           # 21 (all elements)
m.sum(axis=0)      # [5 7 9] column-wise
m.sum(axis=1)      # [6 15] row-wise
m.mean()           # 3.5
m.std()            # standard deviation
m.var()            # variance
m.min(), m.max()   # 1, 6
m.argmin(), m.argmax() # 0, 5 (flat indices)
np.cumsum(m, axis=1) # cumulative sum per row
np.prod(m, axis=0)  # column products
```

5.4 Broadcasting Basics

```
import numpy as np

# Scalar broadcasts to every element
a = np.array([1, 2, 3, 4])
print(a + 100)           # [101 102 103 104]

# 1D added to each row of 2D
m = np.array([[1, 2, 3],
              [4, 5, 6]])
row = np.array([10, 20, 30])
print(m + row)
# [[11 22 33]
#  [14 25 36]]

# Column vector added to each column
col = np.array([[100], [200]])
print(m + col)
# [[101 102 103]
#  [204 205 206]]
```

5.5 Tasks — Operations

Task 5.5.1: Create two 3×3 random arrays A and B. Compute element-wise product, division, and A raised to power B.

Task 5.5.2: Normalise an array of 10 random values (make mean=0, std=1). Verify with `.mean()` and `.std()`.

Task 5.5.3: Given a 4×5 matrix of random integers 1–50, find: overall sum, row-wise max, column-wise mean, and the index of the global minimum.

Task 5.5.4: Use broadcasting to create the multiplication table for numbers 1–9 as a 9×9 matrix. No loops allowed.

Task 5.5.5: Apply `np.sin` and `np.cos` to 100 evenly spaced angles from 0 to 2π . Verify that $\sin^2(x) + \cos^2(x) \approx 1$ for all values.

6. Reshaping, Stacking & Splitting

6.1 Reshape

```
import numpy as np

a = np.arange(24)

# Reshape – must keep total elements same
b = a.reshape(4, 6)
c = a.reshape(2, 3, 4)    # 3D array
d = a.reshape(8, -1)      # -1 auto-computes: 8×3

# ravel vs flatten
flat_view = b.ravel()     # view (fast, changes affect b)
flat_copy = b.flatten()   # copy (safe, independent)

# Adding dimensions
x = np.array([1, 2, 3])
print(x[:, np.newaxis])   # shape (3,1) – column vector
print(x[np.newaxis, :])   # shape (1,3) – row vector
```

6.2 Stacking & Splitting

```
import numpy as np

a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

np.vstack([a, b])          # shape (4,2)
np.hstack([a, b])          # shape (2,4)
np.concatenate([a,b], axis=0) # same as vstack
np.concatenate([a,b], axis=1) # same as hstack
np.stack([a, b], axis=0)    # new axis → (2,2,2)

# Splitting
x = np.arange(12).reshape(4, 3)
parts = np.vsplit(x, 2)     # split into 2 equal halves
np.hsplit(x, 3)             # split into 3 column groups
```

6.3 Tasks — Reshape & Stack

Task 6.3.1: Create a 1D array of 30 elements and reshape it into (5, 6), (2, 3, 5), and (6, 5). Confirm all have 30 elements.

Task 6.3.2: Stack two 3×4 arrays vertically and horizontally. Print the shape of each result.

Task 6.3.3: Create a column vector from [10, 20, 30, 40] using np.newaxis. Add it to a 4×3 matrix using broadcasting.

Task 6.3.4: Create a 6×4 array, split it into 3 equal pieces along the row axis, and print each piece.

7. Comprehensive Challenge Tasks

These tasks integrate multiple concepts from the day. Work through them in order.

7.1 Student Grade Summary

Given marks = `np.array([[72,85,60,78],[88,92,95,90],[45,52,48,55],[65,70,72,68]])` (4 students × 4 subjects), compute: (a) total and average per student, (b) class average per subject, (c) classify each student as Pass (`avg ≥ 50`), Merit (`≥ 65`), or Distinction (`≥ 80`) using `np.where`.

7.2 Sales Data Analysis

Generate a random 12×4 array representing monthly sales of 4 products over 12 months (integers 100–999). Find: (a) best-selling month overall, (b) worst product by total annual sales, (c) months where ALL products exceeded 500 units.

7.3 Temperature Normalisation

Create a 7×24 array representing hourly temperatures for a week (random floats 15–40°C). Normalise each day's temperatures to the range [0, 1] using: `norm = (x - x.min()) / (x.max() - x.min())`. Verify each row's min is 0 and max is 1.

7.4 Custom arange Replacement

Without using `np.arange` or `np.linspace`, build a function `my_range(start, stop, step)` that returns a NumPy array equivalent to `np.arange(start, stop, step)`. Use `np.zeros` and a loop only to fill values, then vectorise it.

7.5 Matrix Statistics Report

Create a 10×10 random float matrix. Print a formatted report showing: shape, dtype, total sum, row-wise means, column-wise standard deviations, global max and its position (row, col), and count of elements greater than 0.5.